第 18 章練習　Linux 系統

本檔只含題目。請先自行建立狀態或變數表，再作答；解答在獨立檔案。

OS18-E01　fork 與 exec

固定 Linux 3.2 的單執行緒 parent：一個已駐留、匿名、private 且可寫的資料頁在 fork 後由父子兩個獨立 mm 映射；在 child 立即 exec 前，child 只讀取該頁，且沒有其他寫入。排除 MAP_SHARED、特殊映射與不繼承的 VMA。

1. COW 如何降低成本？
2. 哪個 task 在 exec 後得到新 program image？
3. 若 exec 失敗，child 應如何處理？

OS18-E02　Module 引用

固定教學 driver：module 已設定 file_operations.owner，1 個 active file handle 持一個保活引用，1 個 queued request／callback 依此 driver 的協定另持引用。沒有其他引用；考慮一般、非強制卸載。

1. 目前可 unload 嗎？
2. 須等哪些事件？
3. 簽章是否能解決 use-after-free？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OS18-E03　VFS 與 page cache

普通檔案 read 命中 page cache 並回傳 4 KiB。比較兩種斷電前路徑：(a) 尚未執行同步；(b) 對該檔案成功完成 fsync。假設檔案系統與裝置正確實作 flush／持久化契約；不考慮不相關檔案。

1. read 成功說明何者可見？
2. 哪些狀態可能未持久？
3. fsync 改變什麼？

OS18-E04　Credentials與regular-file DAC

固定 Linux v3.2 generic_permission 的非 immutable regular file：owner 1000、group 100、mode 0400；caller 的 current_user_ns 等於 inode_userns，相關 capability 在該 namespace 有效。rw mount、無 ACL／LSM 額外拒絕，路徑搜尋已成功；各次獨立要求單一 read、write 或 execute DAC 位元。

1. 比較fsuid1000無cap、fsuid0含CAP_DAC_OVERRIDE、fsuid0無DAC能力、fsuid2000只含CAP_DAC_READ_SEARCH的read/write/execute。
2. mode改0401時，有DAC_OVERRIDE者的execute為何改變？
3. real1000/effective0/saved0先seteuid(1000)，是否永久失去回到0的能力？

OS18-E05　Timeout、EIO與遲到的completion

在固定單一完成通知模型中，R11 generation 3 已 in flight，DMA 指向 F40，總請求 4096 B；先前沒有成功向上層交付或認列的 bytes，錯誤契約回報 transferred=0。裝置可能已向 F40 寫入部分資料；consumer 仍持一個引用。

1. timeout後可立即解除DMA mapping並free F40嗎？給出反例。
2. 裝置確認停止後以EIO完成，通知次數與transferred bytes為何？
3. 相同generation重複completion及舊generation3碰到新generation4時，各應如何處理？
4. terminal callback已完成，consumer引用尚未釋放，F40可否重用？

OS18-E06　Clone sharing與ELF零尾部

初始 parent 是唯一 task、擁有唯一 mm 與 fd table，TID/TGID 均為 100，fd4 指向 open file description O1；無其他共享引用。另給合法對齊且不與其他段重疊的 ELF PT_LOAD：p_offset=0、p_vaddr=0x400000、p_filesz=0x1800、p_memsz=0x3000，page 4 KiB。

1. 普通fork與VM/SIGHAND/THREAD/FILES的thread-like clone，各有幾個task、mm、fd tables與原有open descriptions？
2. 只有THREAD而無SIGHAND、只有SIGHAND而無VM，v3.2如何處理？
3. 逐頁計算ELF file bytes與zero bytes；建立VMA是否表示都resident？
4. exec在flush_old_exec之前與point of no return之後失敗，舊image保證有何差別？

OS18-E07　IRQ重入與module卸載競爭

CPU0 process context 持有 spinlock L，同 CPU interrupt handler 與 CPU1 也遵守同一鎖。自訂 blocking、非 forced 卸載模型：M 每個 handle／request 各持 1 ref，初始恰 3 handles、2 requests 且沒有其他 ref；GOING 不接受新引用，但允許現有工作清理。

1. 只禁止preemption為何可能仍同CPU自我死鎖？
2. 只禁止本CPU IRQ是否足以排除CPU1？應結合哪些機制？
3. 依LIVE→GOING→close handles→finish requests→callbacks靜止列refs與free條件。
4. 為何先檢查refs0再關新入口不安全？

OS18-E08　TCP分段與延遲成因

固定單一 IPv4/TCP trace，MTU 1500、IP/TCP 各 20 B、有效 MSS 恰為 1460 B，無 offload、重傳或額外分段。send 接受 6000 B，指定前四段各 1460 B、末段 160 B；同一段的 enqueue、dispatch、transmit complete、ACK 時刻依序為 0、2、3、13 ms。

1. 計MSS、各wire TCP payload與不含Ethernet/ACK/重傳的L3 bytes。
2. send返回6000能否推出remote app已處理？
3. 分解本trace的queue、device與其後ACK等待時間。
4. 另一筆資料只有queue length20/retrans3/device error1，能否斷定長尾來自queue？需要何種補充？

OS18-E09　Pipe EOF、signal計數與原子semop

fork 後 parent／child 都保留 pipe writer，child 寫入 ABC 後關閉自己的 writer；consumer 持有效讀端並以正 count 讀取。另在同一 pending queue，對同一個被 blocked 的 standard signal 產生 3 次；同組 semaphores 初值 S0=1、S1=0。

1. consumer讀完ABC後是否必有EOF？最後何時read回0？
2. standard signal的3次產生是否必呼叫handler3次？real-time signals有何不同？
3. 要求原子semop批次S0-=1/S1-=1，S1不足時S0是否應先降0？
4. wait queue若先檢查condition再入queue有何漏喚醒；正確協定要補哪一步？

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 18. Linux v3.2 fork, memory, module, DAC, ELF. fork(2), signal(7).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study